



**WYDZIAŁ FIZYKI
I INFORMATYKI STOSOWANEJ**
Uniwersytet Łódzki

Maciej Bujalski

Kierunek: informatyka

Specjalność: informatyka stosowana

Specjalizacja: aplikacje mobilne

Numer albumu: 386012

Rotacyjnie inwariantne sieci neuronowe

Praca magisterska

wykonana pod kierunkiem

dr Krzysztofa Podlaskiego

w Katedrze Systemów Inteligentnych, WFiIS UŁ

Łódź 2025

Spis treści

Wstęp	3
Cel i motywacja pracy	4
Opis pracy	4
Zakres tematyczny	6
Ujęte w zakresie	6
Poza zakresem	6
Artefakty pracy	7
Organizacja pracy	7
Podstawy teoretyczne	8
Wprowadzenie do sieci konwolucyjnych (CNN)	8
Operacja splotu	8
Receptywne pole	9
Nieliniowości i normalizacja	10
Pooling i część klasyfikacyjna	10
Trening	10
Triki architektoniczne	11
Ekwiwariancja translacyjna (kontrast do rotacyjnej)	11
Inwariancja translacyjna i rotacyjna	11
Linear-polar vs. log-polar	12
Problemy z rotacyjną inwariancją w klasycznych CNN	13
Przegląd literatury (E(2) equivariant, CyCNN)	14
CyCNN	14
E(2) equivariant i sieci sterowalne	14
Aspekty implementacyjne i koszt	14
Wnioski w kontekście pracy	15
Opis zbiorów danych	16
MNIST (cyfry odręczne)	16
GTSRB Gray (znaki drogowe w odcieniach szarości)	16
GTSRB RGB (znaki drogowe w kolorze)	18
LEGO (obiekty 3D rzutowane na 2D)	18
Sposób augmentacji danych: zakresy rotacji, łączenie zbiorów	19
Sposób augmentacji danych: rotacje i łączenie zbiorów	19
Rotacje	19
Łączenie zbiorów	19
Organizacja katalogów	20
Scenariusze trenowanie - test	20
Architektury modeli	21
VGG - wariant E (VGG-19, „3×3 everywhere”)	21
ResNet-56 (CIFAR, 6n+2 z n=9)	21
Wersje cykliczne: CyVGG-E i CyResNet-56	22

Uzgodnienia I/O i selektor modeli	22
Standardowe CNN	22
Rotacyjnie inwariantne sieci (CyResNet, CyVGG)	22
Transformacje polarne: linearpolar vs logpolar	23
Implementacja i środowisko eksperymentalne	24
Warstwa CyConv2d (CUDA) oraz jej implementacja	24
Python, PyTorch	25
Struktura projektu	25
Automatyzacja: skrypty trenowania, testowania, ewaluacji	25
Automatyczna optymalizacja hiperparametrów z wykorzystaniem Optuna	26
Obsługa GPU, Docker, WSL	26
Obsługa GPU	27
Docker	27
WSL2	27
Organizacja logów, modeli, confusion matrixów	27
Eksperymenty	30
Scenariusze trenowania/testowania (opis JSON)	30
Pomiar skuteczności (accuracy, macierze pomyłek)	31
Śledzenie metryk: średnia, mediana, odchylenie standardowe	31
Analiza skuteczności względem rotacji	31
Ranking modeli	31
Porównanie wyników	32
CyCNN vs klasyczne CNN	32
VGG vs CyVGG	32
Resnet vs CyResNet	32
CyVGG vs CyResNet	32
Wpływ transformacji (linearpolar vs logpolar)	32
Wydajność na różnych zbiorach	32
Wnioski	33
Skuteczność rotacyjnych architektur	33
Wnioski z automatyzacji i systematyzacji ewaluacji	33
Proponowane dalszych badań	33
Aneks	34
Listingi kodów	34
Dodatkowe wykresy, tablice wyników	34
Bibliografia	35

Wstęp

Obrazy otaczają nas z każdej strony, od zdjęć ze smartfonów, zdjęcia satelitarne, przez monitoring miejski, katalogi produktów i systemy kontroli jakości na liniach produkcyjnych, po systemy wspomagania jazdy. Choć współczesne modele rozpoznawania obrazu radzą sobie bardzo dobrze, w praktyce bywają wrażliwe na pozornie drobne zmiany takie jak obrócenie obiektu o kilkanaście stopni czy niewielki przechył kamery. To, co dla człowieka jest naturalne i natychmiast rozpoznawalne (znak drogowy pod kątem, cyfra obrócona na kartce), dla klasycznej konwolucyjnej sieci neuronowej bywa problemem. Największy problem to brak naturalnej inwariantności względem rotacji, standardowe CNN-y „z definicji” lepiej radzą sobie z przesunięciami niż z obrotami [1], [2].

W ostatnich latach pojawiło się kilka dróg domknięcia tej luki. Jedna to poszerzanie danych o zrotowane przykłady, które poprawiają odporność, ale wydłużają trening i nie gwarantują uogólnienia na wszystkie kąty. Druga to architektury z wbudowaną geometrią: sieci grupowo równoważne (G-CNN, E(2)-equivariant) [3], [4], sieci cykliczne (CyCNN, a w szczególności **CyVGG** i **CyResNet**) operujące na wielu orientacjach oraz przekształcenia do układów polarnych (linear-polar i log-polar), które „prostują” rotacje do przesunięć. Cel jest wspólny, mianowicie by model rozpoznawał „to samo” niezależnie od orientacji, bez agresywnego dublowania danych.

Niniejsza praca skupia się na praktycznej weryfikacji tych podejść. Przygotowano zbiory obejmujące m.in. odręcznie napisane cyfry, znaki drogowe (w kolorze i w odcieniach szarości) oraz syntetyczne obiekty 3D rzutowane na 2D (klocki LEGO), a następnie rozszerzono je o kontrolowane rotacje. Zaimplementowano i porównano wybrane architektury rotacyjnie inwariantne i ich warianty bazowe w **PyTorchu** [5], mierząc wpływ transformacji (linear-polar vs. log-polar), wyboru architektury i zakresu kątów na jakość predykcji. Obliczenia realizowano na kartach graficznych: **NVIDIA GeForce RTX 3070 Ti 8 GB** oraz **RTX 3060 12 GB**, co skróciło czas trenowania i umożliwiło szeroki przegląd eksperymentów. Środowisko uruchomieniowe ustandaryzowano z użyciem **Dockera** dla powtarzalności.

Celem pracy jest nie tylko pokazanie, że „da się” uzyskać odporność na rotacje, ale przede wszystkim wskazanie, **kiedy i jakim kosztem** ją osiągamy oraz które techniki przynoszą największy zysk względem klasycznych CNN-ów, ich wpływ na stabilność i szybkość uczenia, a także które konfiguracje są najpraktyczniejsze w realnych zastosowaniach (OCR, rozpoznawanie znaków, analiza obiektów technicznych). W dalszej części pracy przedstawiono podstawy, dane i augmentację, architektury, środowisko eksperymentalne, protokoły ewaluacji oraz wyniki z analizą i wnioskami.

Cel i motywacja pracy

Konwolucyjne sieci neuronowe (CNN) charakteryzują się zdolnością do analizy obrazów z zachowaniem niezmierności względem translacji. Jednak wciąż brakuje powszechnie uznanych architektur, które zapewniałyby inwariantność względem rotacji. Celem niniejszej pracy jest analiza skuteczności rozwiązań zaproponowanych w literaturze, ukierunkowanych na odporność modeli na rotację danych wejściowych (np. CyCNN [4]).

W ramach pracy zostały przygotowane wzbogacone zbiory danych, obejmujące m.in. zdjęcia odręcznie napisanych liter, znaki drogowe (zarówno w kolorze, jak i w odcieniach szarości) oraz obiekty 3D rzutowane na 2D (klocki LEGO), rozszerzone o kontrolowane rotacje obrazów.

Na podstawie tych zbiorów została przeprowadzona implementacja i ewaluacja wybranych architektur rotacyjnie inwariantnych z wykorzystaniem **PyTorch**. Obliczenia zostały wykonane przy użyciu kart graficznych **NVIDIA GeForce RTX 3070 Ti 8 GB** oraz **RTX 3060 12 GB**, co umożliwiło przyspieszenie trenowania i testowania. Otrzymane wyniki zostały porównane z rezultatami klasycznych sieci konwolucyjnych w celu oceny realnych korzyści z użycia rozwiązań inwariantnych do zbiorów z rotacją.

Opis pracy

Praca magisterska wykorzystuje zaawansowane technologie i narzędzia wspierające badania nad rotacyjnie inwariantnymi sieciami neuronowymi oraz ich zastosowaniem w przetwarzaniu obrazów. W realizacji projektu zastosowano następujące rozwiązania technologiczne:

- **Język programowania Python** - podstawowe narzędzie do implementacji algorytmów oraz obsługi frameworków uczenia maszynowego, dzięki wszechstronności i bogatemu ekosystemowi bibliotek [6] Środowiska uruchomieniowe były izolowane dzięki użyciu `venv`.

Frameworki uczenia maszynowego:

- **PyTorch** - elastyczny framework do budowy, trenowania i wdrażania modeli ML/DL (w tym własnych warstw, takich jak `CyConv`) [7].
- **Optuna** - biblioteka do automatycznej optymalizacji hiperparametrów (*HPO*) z obsługą efektywnych strategii próbkowania (**TPE**) oraz mechanizmów wczesnego przerywania treningów (**MedianPruner** itp.). Umożliwia ona definiowanie przestrzeni poszukiwań, rejestrowanie metryk, zapisywanie wyników (np. do CSV/JSON) oraz łatwe odtwarzanie najlepszych konfiguracji w postaci *study*. Integracja z PyTorchem odbywa się bez zmian w architekturze modeli i pozwala skrócić czas eksperymentów bez utraty jakości [8].
- **Modele cykliczne (CyCNN)**. W pracy zostało przyjęte podejście, w którym obraz został przemapowany do współrzędnych (ρ, φ) . Dzięki temu obrót R_α staje się przesunięciem o α po osi φ . Konwolucje zostały zastąpione warstwami cylindrycznymi (`CyConv`) z cyklicznym paddingiem po φ . Dla każdego filtra zostało przygotowanych n orientacji, a odpowiedzi zostały złożone z dodatkową osią „orientacja”. Obrót wejścia powoduje cykliczne przesunięcie po tej osi, a pooling po orientacjach daje inwariantność względem rotacji. Został ustawione takie parametry jak stały środek układu polarnych, stała siatka próbkowania oraz biliniarna

interpolacja. Padding po φ został ustawiony na cykliczny. Implementacja została wykonana w **PyTorchu** (punkt odniesienia: CyCNN [4]).

- **Wykorzystanie akceleracji GPU (NVIDIA)** - obliczenia zostały znacząco przyspieszone dzięki użyciu kart **RTX 3070 Ti 8 GB** oraz **RTX 3060 12 GB**. Frameworki takie jak PyTorch wspierają **CUDA**, **Tensor** oraz **cuDNN**, co umożliwia efektywne wykorzystanie zasobów GPU [9], [10]. Monitorowanie i diagnostyka zostały wykonane z użyciem narzędzia `nvidia-smi`.
- **Rdzenie CUDA (CUDA Cores)**. Podstawowe jednostki wykonawcze multiprocesorów strumieniowych (SM) realizują obliczenia arytmetyczne w precyzji **FP32/INT32**. Konwolucje oraz mnożenia macierzy są wykonywane na rdzeniach CUDA zawsze wtedy, gdy nie są aktywowane Tensor Cores (np. czysty FP32 bez **TF32/AMP**). Wydajność zależy od obsadzenia SM-ów (occupancy), doboru rozmiaru bloków (wielokrotność 32 wątków — **warp**), koalescencji dostępu do pamięci globalnej oraz wykorzystania pamięci współdzielonej. Warstwa `CyConv2d` wymusza `contiguous()` i obecność tensora na CUDA przed wywołaniem jądra; duży workspace sprzyja kafelkowaniu i ogranicza liczbę odczytów z DRAM, co poprawia przepływ danych na SM-ach [9].
- **Tensor Cores (Ampere)**. Zastosowane karty graficzne RTX (3070 Ti, 3060) mają rdzenie Tensor, które sprzętowo przyspieszają operacje macierzowe (konwolucje/matmul). Biblioteki **cuDNN/cuBLAS** na architekturze **Ampere** domyślnie mogą używać trybu **TF32** dla obciążeń FP32, co daje dodatkowe przyspieszenie bez zmian w modelu. Dodatkowo, w **PyTorchu** możliwe jest włączenie **mieszanej precyzji** (FP16/BF16) przez **AMP** w miejscach, gdzie to bezpieczne, przy włączeniu tego feature, zwykle przyspiesza to trening przy porównywalnej jakości (szczegóły znajdują się w dokumentacji). [11], [12], [13], [14]
- **System operacyjny: Linux (Ubuntu LTS)**. Główne środowisko uruchomieniowe stanowił system operacyjny **Ubuntu** (dystrybucja LTS) posiadający stabilne jądro, pakiety z APT, łatwą integrację ze sterownikami NVIDIA i CUDA. Treningi uruchamiane były **lokalnie** na maszynach z GPU NVIDIA. [15]. Dla zgodności ze środowiskami Windows używano też wariantu **WSL2** (ten sam obraz Dockera i ta sama konfiguracja) [16].
- **Konteneryzacja za pomocą Dockera** - odizolowane środowiska uruchomieniowe ułatwiły replikację i współdzielenie projektu, także z obsługą GPU [17].

Zakres tematyczny

Niniejsza praca dotyczy odporności modeli klasyfikacji obrazów na rotacje w płaszczyźnie 2d. Skupiono się na porównaniu klasycznych architektur z ich wersjami rotacyjnie inwariantnymi oraz na wpływie przekształceń polarnych na jakość predykcji. Badania zostały przeprowadzone na obrazach 2D i ich rotacjach planarnych. W szczególności zostały porównane warianty bazowe **VGG/ResNet** z wersjami cyklicznymi **CyVGG/CyResNet**, a także wpływ mapowania **linear-polar** i **log-polar**. Eksperymenty zostały wykonane na zbiorach **MNIST**, **GTSRB_gray**, **GTSRB_RGB** i **LEGO** z kontrolowanymi rotacjami.

Ujęte w zakresie

- **Architektury modeli:** zostały zaimplementowane i porównane warianty bazowe **VGG** oraz **ResNet** [18], [19], a także ich wersje cykliczne **CyVGG** i **CyResNet** (modele rotacyjnie inwariantne) [4].
- **Przekształcenia polarne:** została oceniona użyteczność mapowania **linear-polar** oraz **log-polar** jako etapów wstępnego przetwarzania służących „prostowaniu” rotacji do przesunięć [4], [20].
- **Zbiory danych:** zostały wykorzystane następujące zbiory: **MNIST** (odręczne cyfry, 28x28 → 32x32, grayscale) [21], **LEGO** (syntetyczne obiekty 3D rzutowane na 2D, 96x96, grayscale), **GTSRB** (znaki drogowe, 32x32, grayscale) oraz **GTSRB_RGB** (wersja kolorowa) [22]. Zbiory zostały rozszerzone o kontrolowane rotacje oraz zostały przygotowane spójne podziały zbiorów na train/val/test.
- **Augmentacja i protokół nauki, validacji oraz testów:** zostały zdefiniowane zakresy kątów do sprawdzenia, liczba treningów, podział na zbiory train/val/test (uczący/ walidacyjny/testowy), z możliwością powtórzenia trenowań.
- **Środowisko i implementacja:** został wykorzystany **PyTorch** z akceleracją **CUDA/cuDNN** na kartach **NVIDIA GeForce RTX 3070 Ti 8 GB** oraz **NVIDIA GeForce RTX 3060 12 GB** [7], [9], [10]. Przygotowane zostały skrypty w Pythonie do trenowania, testowania i ewaluacji [6]. Środowisko uruchomieniowe zostało ustandaryzowane z użyciem **Dockera** [17].
- **Metryki i analiza:** została przeprowadzona ocena jakości (accuracy, macierze pomyłek), analiza stabilności (średnia/mediana/odchylenie standardowe), wpływ kąta rotacji na skuteczność oraz koszt obliczeniowy (czas trenowania, rozmiar modelu).

Poza zakresem

- Detekcja obiektów i segmentacja - w pracy rozpatrywana jest wyłącznie klasyfikacja [23], [24].
- Inwariancja względem skali, ścinania i pełnych przekształceń afinicznych - analizowana jest tylko rotacja w płaszczyźnie [25], [26].

- Rotacje w geometrii 3D oraz zagadnienia widzenia stereo - pozostają poza zakresem [27], [28].
- Trening na bardzo dużych korpusach z pre-treningiem self-supervised oraz szerokim AutoML/hyper-search - nie został realizowany [29], [30], [31].
- Odporność na silne zakłócenia (szum, okluzje) - poza zakresem; skupiono się wyłącznie na rotacji [32], [33].

Artefakty pracy

- Repozytoria z kodem, skryptami i plikami konfiguracyjnymi.
- Pliki konfiguracyjne eksperymentów i opis danych.
- Wytrenowane wagi modeli (wybrane checkpointy) oraz raporty z ewaluacji.
- Tekst pracy z dokumentacją eksperymentów i wnioskami.

Organizacja pracy

Struktura pracy została ułożona tak, by od podstaw przejść do wyników. W rozdziale **Podstawy teoretyczne** zostały zebrane pojęcia i narzędzia: CNN, inwariancja/ekwawariancja, przekształcenia polarne oraz prace pokrewne (G-CNN, E(2)-equivariant, CyCNN). W **Opisie zbiorów danych** zostały przedstawione MNIST, LEGO, **GTSRB_gray** i **GTSRB_RGB** oraz sposób augmentacji (rotacje, podziały train/val/test). W **Architekturach modeli** zostały opisane warianty bazowe **VGG/ResNet** oraz ich wersje cykliczne **CyVGG/CyResNet**, wraz z transformacjami linear-polar / log-polar. Rozdział **Implementacja i środowisko** zawiera szczegóły techniczne: **PyTorch**, **CUDA** i **cuDNN** (RTX 3070 Ti 8 GB, RTX 3060 12 GB), **Docker**, strukturę projektu i skrypty. W **Eksperymentach** zostały zdefiniowane scenariusze, metryki i sposób ewaluacji. Dalej, w **Porównaniu wyników**, zostały zestawione modele (VGG vs. CyVGG, ResNet vs. CyResNet, wpływ transformacji) i omówiona stabilność/czas. Na końcu sekcja **Wnioski** zbierają najważniejsze **obserwacje** i wskazuje kierunki dalszych badań. Sekcja **Aneks** zawiera kody i dodatkowe wykresy.

Podstawy teoretyczne

Celem tego rozdziału jest uporządkowanie pojęć, które są potrzebne do zrozumienia dalszych eksperymentów. Najpierw zostały przedstawione podstawy klasycznych sieci konwolucyjnych takich jak: idea splotu, lokalne pola recepcyjne, współdzielenie wag i wynikająca z tego ekwiwariancja względem translacji [1], [2], [34]. Następnie omawiane są parametry geometrii warstwy (stride, padding, dylacja), zależności rozmiarów wejścia i wyjścia oraz sposób, w jaki pooling buduje praktyczną inwariancję na przesunięcia.

Następnie przedstawiona zostaje różnica między **ekwiwariancją** a **inwariancją** oraz pokazane są ograniczenia klasycznych CNN w kontekście rotacji. Ten brak zgodności grupowej dla obrotów motywuje dwie ścieżki rozwijane w literaturze, pierwsza to mapowanie do układu biegunowego i operowanie na osi kąta w sposób cykliczny (linia **CyCNN**), zaś druga to konstrukcje oparte o sploty grupowe i jądra sterowalne w grupie $E(2)$ (sieci **E(2)-equivariant**) [3], [4], [35]. W tej pracy wykorzystywana jest pierwsza ścieżka, ponieważ pozwala zachować standardowy pipeline i porównywalny budżet parametrów, a jednocześnie wprowadzić kontrolowaną ekwiwariancję rotacyjną, która po agregacji orientacji przechodzi w inwariancję.

Dla kompletności omówione zostaną też praktyczne aspekty przekształceń polarnych (linear-polar i log-polar), sposób liczenia pól recepcyjnych po takich mapowaniach oraz wpływ decyzji implementacyjnych (interpolacja, wybór środka, cykliczny padding po φ) na stabilność uczenia. Taki zestaw podstaw pozwala czytelnie oddzielić wpływ **augmentacji** od wpływu **architektury** i stanowi fundament pod analizę wyników w dalszych rozdziałach.

Wprowadzenie do sieci konwolucyjnych (CNN)

Sieci konwolucyjne (CNN) zostały zaprojektowane do pracy na danych o strukturze siatkowej lub macierzowej, takich jak obrazy dwuwymiarowe (2D). Ich kluczowe cechy to **lokalne receptywne pola**, **współdzielone wagi** oraz **operacja splotu**. Dzięki temu możliwe jest skalowanie modeli na duże obrazy oraz lepsze uogólnianie niż w przypadku sieci posiadającej pełne połączenia.

Zamiast analizować cały obraz jednocześnie, CNN wykorzystują mały filtr, który przesuwa się po lokalnych fragmentach obrazów. W ten sposób uczą się prostych detektorów (np. krawędzi, tekstur, kształtów), zaś w wyższych warstwach bardziej złożonych cech [1], [34].

Dla przesunięcia $\mathcal{T}_t$ oraz jądra K zachodzi własność **ekwiwariancji translacyjnej**:

$$\mathcal{T}_t(X) * K = \mathcal{T}_t(X * K).$$

Intuicyjnie oznacza to, że jeśli obraz zostanie przesunięty, to odpowiadająca mu mapa cech również przesunie się o ten sam wektor. **Inwariancja na przesunięcia** osiągnięta jest w praktyce poprzez pooling (lokalny lub globalny) bądź zwiększenie kroku (stride). Rzadsze próbkowanie powoduje, że wynik klasyfikacji nie jest zależny od dokładnej pozycji obiektu [1], [2].

Operacja splotu

Intuicyjnie ten sam mały filtr „przesuwa się” po obrazie i oblicza ważoną sumę pikseli. Dzięki temu uzyskuje się **współdzielenie wag** (liczba parametrów nie rośnie wraz z H, W) oraz **lokalność**

obliczeń [1], [2].

Kształty. Wejście: $X \in \mathbb{R}^{C_{in} \times H \times W}$. Zestaw jąder: $K \in \mathbb{R}^{C_{out} \times C_{in} \times k \times k}$. Wyjście: $Y \in \mathbb{R}^{C_{out} \times H' \times W'}$.

Definicja (pojedynczy kanał wyjściowy c):

$$Y_c(u, v) = \sum_{i=1}^{C_{in}} \sum_{a,b} K_{c,i}(a, b) X_i(u - a, v - b).$$

W praktyce większość frameworków oblicza **korelację krzyżową** (bez odwracania jądra), pomimo tego, że w API funkcja nazywana jest `conv` [2]. Nie ma to jednak końcowo znaczenia dla procesu uczenia, bo sieć i tak ostatecznie dobierze właściwe wagi.

Stride, padding, rozmiary Parametry „geometrii” warstwy:

- **padding** p - ile pikseli dodajemy na brzegach;
- **stride** s - co ile pikseli przesuwamy okno;
- **dylacja** d - „rozciąga” jądro poprzez wstawienie przerw między próbkami.

Same/valid/stride, a ekwiwariancji

Dokładna ekwiwariancja translacyjna zachodzi przy splocie bez zmiany rozmiaru. W praktyce **padding „same”, stride > 1 i pooling** wprowadzają drobne odchylenia (aliasing na siatce próbkowania), co obniża „idealność” ekwiwariancji, efekt ten jest znany i opisywany w literaturze [2], [36].

Rozmiar wyjścia (dla jądra $k \times k$):

$$H' = \left\lfloor \frac{H + 2p - d(k - 1) - 1}{s} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W + 2p - d(k - 1) - 1}{s} \right\rfloor + 1.$$

Typowe ustawienia. - *valid*: $p = 0$ - mapy cech maleją; - *same* (dla $s = 1$): $p = \lfloor k/2 \rfloor$ - $H' = H$, $W' = W$; - *stride* > 1 : wbudowane **podpróbkowanie** (mniej obliczeń, mniejsza rozdzielczość); - *dylacja* > 1 : większe **efektywne** pole widzenia bez nowych parametrów (częste w detekcji/segmentacji) [2].

Receptywne pole

Receptywne pole to fragment obrazu „widziany” przez daną aktywację w mapie cech. Im głębsza warstwa, tym pole jest większe, ponieważ kolejne sploty agregują informacje z coraz większych fragmentów wejścia. Dla jąder k_ℓ i kroków s_ℓ zachodzi:

$$R_1 = k_1, \quad R_\ell = R_{\ell-1} + (k_\ell - 1) \prod_{j < \ell} s_j.$$

W praktyce nie wszystkie piksele w obrębie tego pola mają taki sam wpływ. Największy wpływ ma obszar centralny, a znaczenie maleje ku brzegom, przy czym rozkład wpływu przypomina rozkład

Gaussa. Z tego powodu w bazowych architekturach VGG i ResNet dobiera się głębokość tak, aby przy stosowanej rozdzielczości objąć cały obiekt bez utraty istotnego kontekstu [37].

W wariantach **CyCNN** po przejściu do współrzędnych (ρ, φ) oraz przy **cyklicznym paddingu** wzdłuż φ sieć „widzi” pełny zakres orientacji bez artefaktów brzegowych, co ułatwia stabilne uczenie cech niezależnych od kąta [4].

Receptywne pole w układzie polarnym

Po mapowaniu $(x, y) \rightarrow (\rho, \varphi)$ receptywne pole staje się „wąskim paskiem” wzdłuż promienia ρ i przy czym stabilnym po kącie φ . Dzięki temu obrót $\mathcal{R}_\alpha$ na wejściu jest równoważny **przesunięciu** o α po osi φ . Warstwy typu CyConv z **cyklicznym paddingiem** wzdłuż φ nie „tną” informacji na brzegach, co oczywiście wzmacnia ekwiwariancję rotacyjną [4].

Nieliniowości i normalizacja

Blok konwolucyjny pozostaje **taki sam** we wszystkich wariantach (bazowych i cyklicznych). Celem porównania jest wpływ **rotacji**, a nie dobór aktywacji.

Zastosowano standardową normalizację, celem stabilizacji uczenia. W wariantach **CyCNN** statystyki normalizacji liczone są **wspólnie po osi orientacji**, tak aby **nie faworyzować żadnego kąta** i nie naruszać własności rotacyjnych [4], [38].

Pooling i część klasyfikacyjna

W modelach **CyCNN** inwariancja względem rotacji uzyskiwana jest przez **agregację po orientacjach** (pooling po osi kątów). Następnie, we wszystkich modelach, stosowany jest **global average pooling (GAP)** oraz pojedyncza warstwa liniowa w klasyfikatorze. GAP redukuje liczbę parametrów i zmniejsza zależność od położenia w obrębie map cech [39]. Część klasyfikacyjna pozostaje taka sama w wariantach bazowych i cyklicznych, aby izolować wpływ części „rotacyjnej” [4].

Pooling po orientacjach - szczegóły praktyczne

Agregacja po osi *orientacja* (avg lub max) realizuje **inwariancję rotacyjną**. Na wynik końcowy wpływa liczba orientacji **n**, czyli większe **n** oznacza dokładniejszą rozdzielczość kątową (mniejszy błąd zaokrąglenia $2\pi/n$), ale też wyższy koszt obliczeń i pamięci. W eksperymentach utrzymano identyczny klasyfikator za poolingiem, aby jednoznacznie zmierzyć wpływ części „rotacyjnej” [4].

Trening

Protokół trenowania został **zamrożony** między wariantami (te same: liczba epok, rozmiar batcha, budżet obliczeniowy, wczesne zatrzymanie - wszystko to zgodnie z planem eksperymentu). Augmentacje ograniczono do tych **niezależnych od rotacji** w testach „czysto architektonicznych”. Augmentację rotacją stosowano wyłącznie w eksperymentach kontrolnych, tak aby pokazać różnicę między „augmentacją” a „architekturą”.

Triki architektoniczne

Jako bazy zastosowano VGG-19 (bloki 3×3) i ResNet-56 w wariacie CIFAR (bloki 3×3). W wersjach cyklicznych każdą warstwę Conv2d zastąpiono CyConv2d. Po stronie geometrii zastosowano cykliczny padding wzdłuż osi kątowej φ . Układ bloków, liczby kanałów, BN/ReLU, GAP i klasyfikator zostały pozostawione bez zmian, tak aby utrzymać porównywalny budżet parametrów/FLOPs względem bazowych modeli. Na poziomie definicji modeli nie dodano jawnej osi „orientacja” ani osobnego pooling po orientacjach, jeśli mechanizmy rotacyjne są użyte, są one enkapsulowane w implementacji CyConv2d (jądro CUDA), a nie w topologii sieci. Nie zostały prowadzone modyfikacje niezwiązane z rotacją (np. zmiana funkcji aktywacji, normalizacja, głębokość, rozmiar jąder, liczby kanałów czy regularizacji), aby nie mieszać ich wpływu z efektem komponentu rotacyjnego[4].

Ekwiwariancja translacyjna (kontrast do rotacyjnej)

Dla przesunięcia $\mathcal{T}_t$ i splotu zachodzi:

$$\mathcal{T}_t(X) * K = \mathcal{T}_t(X * K),$$

co tłumaczy, dlaczego klasyczne CNN dobrze radzą sobie z translacją [2]. Brak analogicznego mechanizmu dla rotacji motywuje użycie przekształceń polarnych i/lub modeli cyklicznych w dalszej części.

Ekwiwariancja rotacyjna w dyskretnej grupie C_n

Dla indeksu orientacji $k \in \{0, \dots, n-1\}$ i kąta $\theta_k = \frac{2\pi k}{n}$ ekwiwariancję można zapisać jako

$$\Phi(\mathcal{R}_{\theta_k} X)[\cdot, m] = \Phi(X)[\cdot, m+k \bmod n],$$

czyli **obrót wejścia = cykliczne przesunięcie po osi orientacji**. Następnie pooling po tej osi znosi zależność od kąta (inwariancja) [4].

Ekwiwariancja (przewidywalna zmiana reprezentacji):

$$\Phi(\mathcal{T}_t X) = \Pi_t \Phi(X), \quad \Phi(\mathcal{R}_\alpha X) = \Pi_\alpha \Phi(X),$$

gdzie Π to działanie grupy na cechach (dla translacji - przesunięcie map, dla rotacji - np. **cykliczny shift** po osi „orientacja”) [1], [35].

Inwariancja translacyjna i rotacyjna

Niech $\mathcal{T}_t$ oznacza przesunięcie o wektor t , $\mathcal{R}_\alpha$ - obrót o kąt α , a Φ - mapę cech / model.

Inwariancja (wynik nie zależy od transformacji):

$$\Phi(\mathcal{T}_t X) = \Phi(X), \quad \Phi(\mathcal{R}_\alpha X) = \Phi(X).$$

W praktyce: - **translacja**: uśrednianie / podpróbkiwanie (GAP, stride); - **rotacja**: (a) augmentacja o obroty, (b) architektura ze śledzeniem orientacji (oś „kąt” + pooling po orientacjach), (c) mapowanie do współrzędnych polarnych, gdzie obrót staje się przesunięciem po osi φ [2], [4], [20].

Linear-polar vs. log-polar

- **Linear-polar:** równy przyrost w pikselach po ρ i φ . Stabilne kąty, prosta implementacja. Rozwiązanie to jest dobre pod **inwariancję rotacyjną**.
- **Log-polar:** skala rośnie logarytmicznie po ρ - zmiany skali stają się przesunięciami po osi promienia. Jest to idealne rozwiązanie pod **rotacje i skale**, ale bliżej środka rośnie gęstość próbkowania i przy tym wrażliwość na wybór środka [20].

W praktyce stosowana jest interpolacja biliniarna wraz z **cyklicznym paddingiem** po φ , oraz stałym środkiem. Blisko $\rho=0$ warto wygładzić / wykluczyć kilka próbek, aby uniknąć „osobliwości” środka [4].

Problemy z rotacyjną inwariancją w klasycznych CNN

W praktyce klasyczne CNN dobrze znoszą przesunięcia, ale nie domykają symetrii obrotu. Wynika to z natury splotu na dyskretnej siatce, efektów interpolacji oraz braku jawnej reprezentacji kąta w strumieniu cech. Poniżej zebrano najważniejsze źródła nieinwariancji, które bezpośrednio wpływają na wyniki i ich interpretację.

- **Kierunkowość filtrów.** Małe jądra 3×3 i 5×5 reagują głównie na jedną orientację. Aby pokryć wiele kątów, sieć musiałaby nauczyć się wielu obróconych kopii tych samych detektorów, co zwiększa zapotrzebowanie na dane i parametry. Kompozycja kilku warstw częściowo pomaga, ale bez mechanizmów ukierunkowanych na kąt problem nie znika.
- **Augmentacja nie domyka całości.** Obracanie wzbogaca dane, ale pokrywa tylko zbiór dyskretnych kątów. Między tymi wartościami pozostaje „szczelina” generalizacji, zwłaszcza przy rzadkiej siatce kątów i ograniczonym budżecie. Dodatkowo augmentacja wydłuża trening i wnosi wariancję związaną z losowym próbkowaniem kątów.
- **Aliasing i interpolacja.** Obrót danego rastra wymaga resamplingu i doboru jądra interpolacji. Pojawia się wtedy rozmycie lub aliasing, a wysokie częstotliwości są tłumione inaczej zależnie od kąta oraz implementacji [36]. Skutkiem jest niespójność odpowiedzi nawet przy niewielkich obrotach tego samego obiektu.
- **Krawędzie i padding.** Dopełnianie „same/zero” łamie symetrię przy brzegach. W pobliżu krawędzi zmienia się kontekst, więc odpowiedzi nie są idealnie ekwiwariantne. Stride i pooling pogłębiają ten efekt przez rzadsze próbkowanie i aliasing, co dodatkowo obniża stabilność na małe obroty [2].
- **Brak osi orientacji.** W typowych CNN nie zapisuje się jawnie informacji o kącie wykrytej cechy. Orientacje „mieszają się” w kanałach, więc późniejsze domknięcie do inwariancji (np. przez pooling) nie ma do czego się odnieść. Stąd potrzeba osi „orientacja” i operacji cyklicznych lub mapowania do układu polarnego.
- **Brak zgodności grupowej.** Standardowy splot gwarantuje ekwiwariancję dla translacji, ale nie dla rotacji. Na siatce pikseli obrót nie komutuje ze splotem jak przesunięcie. Klasyczne CNN nie mają więc gwarancji, że $\Phi(\mathcal{R}_\alpha X)$ jest prostą transformacją $\Phi(X)$.
- **Wczesne warstwy i pole widzenia.** We wczesnych warstwach receptywne pole jest małe, przez co lokalne rotacje bywają nierozróżnialne od innych zmian. Szerszy kontekst pojawia się dopiero po poolingach i podpróbkowaniu, co jednocześnie obniża precyzję kątową.
- **Interakcja z rozmiarem i kształtem obiektu.** Rotacja zmienia relacje między detalami, a siatką próbkowania (np. inna liczba „przecinanych” pikseli wzdłuż krawędzi przy różnych kątach). Skutkiem są fluktuacje aktywacji i decyzji zależne od rasteryzacji i kąta, a nie od samej klasy.

Przegląd literatury (E(2) equivariant, CyCNN)

Literatura o sieciach ekwiwariantnych rozwija się zasadniczo w dwóch kierunkach. Pierwszy nurt to podejścia o charakterze geometrycznym, które mapują obraz do współrzędnych biegunowych i traktują oś kąta jako wymiar cykliczny. Drugi nurt to modele o ściśle zdefiniowanej ekwiwariancji względem grupy przekształceń $E(2)$, budowane są one poprzez sploty grupowe oraz jądra sterowalne projektowane zgodnie z reprezentacjami grupy. Oba podejścia dążą do reprezentacji odpornej na obrót. Różnią się jednak stopniem formalizacji, kosztem obliczeniowym i wysiłkiem inżynierskim potrzebnym do integracji z typowymi pipeline'ami.

CyCNN

W rodzinie CYCNN obraz przemapowywany jest do układu (ρ, φ) tak, aby obrót w płaszczyźnie stał się przesunięciem po osi φ . Przetwarzanie odbywa się warstwami cylindrycznymi z dopełnieniem cyklicznym wzdłuż osi danego kąta. Dla każdego filtra przyjmuje się dyskretny zbiór orientacji opisany grupą C_n . Obrót wejścia z tego zbioru przekłada się na cykliczne przesunięcie po wymiarze orientacji w mapach cech. Agregacja po orientacjach domyka inwariancję. Wybór liczby orientacji równoważy rozdzielczość kątową i koszt obliczeń. Istotne są także decyzje implementacyjne takie jak wybór środka, interpolacja przy odwzorowaniu oraz właściwe domknięcie brzegu dla $\varphi = 0$ i $\varphi = 2\pi$, tak aby uniknąć artefaktów. Zaletą CyCNN jest zgodność z klasyczną praktyką tworzenia modeli. Zamiana klasycznej konwolucji na odpowiednik cylindryczny odbywa się bez zmiany interfejsu warstwy, co ułatwia kontrolowane porównania z wersjami bazowymi przy podobnym budżecie parametrów i zapotrzebowaniu na moc obliczeniową (FLOPs) [4].

E(2) equivariant i sieci sterowalne

Druga linia prac zaś nie zmienia układu współrzędnych, lecz definiuje spłot bezpośrednio na grupie $E(2)$ albo na przestrzeniach jednorodnych tej grupy. Filtry konstruowane są w zgodzie z reprezentacjami, co pozwala dzielić wagi pomiędzy orientacje oraz zapewniać ekwiwariancję także dla kątów traktowanych jako wielkości ciągłe. W literaturze opisano zarówno wersje dyskretne w stylu G-CNN, jak i konstrukcje sterowalne, w których filtry rozwija się w bazach harmonicznym i ogranicza regułami reprezentacji [3], [40], [41]. Rozszerzenia obejmują również grupy dihedralne D_n , które pozwalają modelować rotacje i odbicia, a także modele na przestrzeniach jednorodnych, co ułatwia precyzyjne wskazanie, gdzie ma zajść ekwiwariancja, a gdzie inwariancja.

Aspekty implementacyjne i koszt

Modele oparte na formalizmie $E(2)$ zapewniają ściśle gwarancje wynikające z algebry grupy oraz konstrukcji jąder. Osiąga się to kosztem większych wymagań obliczeniowych i pamięciowych oraz bardziej złożonej implementacji. Pojawia się konieczność definiowania typów pól cech, respektowania ograniczeń na kształt filtrów i pracy w bazach harmonicznym. Linia CyCNN jest lżejsza wdrożeniowo, ponieważ wystarcza zastąpić standardowy operator splotu jego wariantem cylindrycznym i traktować wymiar kąta jako cykliczny. Dokładność ekwiwariancji zależy tu od liczby rozpatrywanych orientacji, jakości interpolacji oraz stabilnego wyboru środka. W zamian zachowana jest zgodność z istniejącymi modelami VGG i ResNet oraz ze standardowymi komponentami, takimi jak BatchNorm, dropout i GAP.

Wnioski w kontekście pracy

Wybrana została architektura z rodziny CyCNN, ponieważ ułatwia porównanie z modelami bazowymi i pozwala kontrolować informację o orientacji bez ingerencji w pozostałe elementy sieci. Mapowanie do (ρ, φ) oraz cykliczne traktowanie osi kąta umożliwiają zrealizowanie ekwiwariancji na etapie ekstrakcji cech, a agregacja po orientacjach zamienia ją w inwariancję. Taki układ sprzyja rzetelnemu porównaniu augmentacji rotacją z architekturą z wbudowaną obsługą rotacji przy tym samym klasyfikatorze i zbliżonym budżecie parametrów modeli [3], [4], [40], [41].

Opis zbiorów danych

W pracy wykorzystano cztery zbiory: MNIST, GTSRB (w dwóch wariantach: Gray i RGB) oraz LEGO. Wszystkie dane zostały ujednolicone pod kątem rozdzielczości i kanałów oraz znormalizowane per kanał. MNIST przeskalowano do 32×32 w skali szarości o 10 klasach [34], zaś GTSRB Gray do 32×32 w skali szarości mający 43 klasy, a GTSRB RGB również przeskalowano do 32×32 z trzema kanałami (też 43 klasy), z zachowaniem oficjalnego podziału na trening i test (IJCNN 2011) [22], [42]. Zbiór LEGO przygotowano jako obrazy 96×96 w skali szarości posiadający 50 klas [43]. Część walidacyjną zbiorów wydzielono z części treningowej we wszystkich zbiorach, pozostawiając test jak w oryginale. Ujednolicenie rozmiaru wejścia i części klasyfikacyjnej pozwala porównywać modele bazowe i cykliczne przy tym samym budżecie obliczeń. Szczegóły formatów (IDX/NPY), normalizacji oraz scenariuszy rotacyjnych zostały opisane rozdziałach poświęconych augmentacji i implementacji.

MNIST (cyfry odręczne)

Zbiór MNIST to klasyczny benchmark rozpoznawania cyfr 0-9 [34]. Obejmuje **60 000** próbek uczących i **10 000** testowych. Obrazy mają rozdzielczość 28×28 , są w skali szarości, a wartości pikseli mieszczą się w zakresie $[0, 255]$. W eksperymentach wartości te są najpierw skalowane do $[0, 1]$, a następnie standaryzowane per kanał. Szczegóły formatu i struktury plików są dostępne na stronie projektu [44].

Na potrzeby porównań obrazy zostały **przeskalowane do 32×32** , aby dopasować je do ustawień „cifarowych” stosowanych w VGG i ResNet. Wejście ma **1 kanał**, a liczba klas wynosi **10**. Normalizacja jest liczona na zbiorze uczącym, przy czym w praktyce często przyjmuje się wartości z przykładów referencyjnych PyTorch: średnia ≈ 0.1307 i odchylenie standardowe ≈ 0.3081 [45]. Podział na zbiory utrzymuje spójność z resztą eksperymentów: z części treningowej wydzielany jest zbiór walidacyjny (5 000 próbek), a test pozostaje jak w oryginale.

Wybór MNIST wynika z jego prostoty i „czystości”, co pozwala szybko iterować i w kontrolowany sposób badać wpływ **rotacji cyfr**. Zbiór dobrze nadaje się do uczciwego porównania modeli bazowych (VGG/ResNet) z **wersjami cyklicznymi** (CyVGG/CyResNet) przy tym samym budżecie obliczeń. Rotacje ujawniają też naturalne przypadki brzegowe, np. pary **6/9** czy **2/5**, które przy większych kątach bywają mylone, pozwala to wyraźniej odróżnić wpływ **augmentacji** od wpływu **architektury**.

W części poświęconej augmentacji wprowadzane są kontrolowane scenariusze kątowe: wariant **bez rotacji** jako punkt odniesienia, warianty z **małymi i średnimi obrotami**, a także **pełny zakres 0-360°**. Celem jest wykazanie, kiedy **architektura cykliczna** zapewnia przewagę nad samą augmentacją rotacją.

GTSRB Gray (znaki drogowe w odcieniach szarości)

German Traffic Sign Recognition Benchmark (GTSRB) to zestaw znaków drogowych z rzeczywistych nagrań, obejmujący **43 klasy**, z oficjalnym podziałem na część uczącą i testową (IJCNN

2011) [22], [42]. W literaturze często przytaczana jest również analiza „man vs. computer” z metrykami porównawczymi [46].

W wariancie **Gray** zastosowanym w tej pracy wszystkie obrazy zostały **przeskalowane do 32×32 i skonwertowane do skali szarości** (1 kanał), tak aby dopasować je do ustawień „cifarowych” oraz wyizolować wpływ **rotacji** od informacji barwnej. Zachowano **43 klasy**; walidację wydzielono z **oficjalnej** części treningowej (spójnie z pozostałymi zbiorami). Zastosowano **normalizację per-kanał** wyliczaną na zbiorze uczącym.

Wybór wersji w odcieniach szarości motywowany jest tym, że kolor bywa silną wskazówką (np. czerwone obramowania, niebieskie tła), podczas gdy celem jest tu głównie **geometria** i ocena, co daje **architektura rotacyjnie inwariantna** na tle bazowej, bez „pomocy” informacji barwnej. Taki wariant ułatwia też czyste porównania z **GTSRB RGB** (sekcja poniżej), w których różnice można przypisać właśnie dostępności koloru.

Zbiór GTSRB stawia kilka typowych wyzwań: nierównomierny rozkład klas, duża zmienność skali i oświetlenia, efekty perspektywy oraz rozmycie w ruchu. Te czynniki utrudniają proste uogólnianie i dobrze testują **stabilność względem rotacji** [22], [46].

W eksperymentach wykorzystano scenariusze kątowe opisane w rozdziale *Augmentacja i protokół*: wariant **bez rotacji** (baseline), zestawy **małych/średnich obrotów**, połączenia **różnych** kombinacji kątów oraz **pelen zakres $0-360^\circ$** . Pozwala to porównać **VGG/ResNet** z **CyVGG/CyResNet** przy identycznym budżecie obliczeń.

GTSRB RGB (znaki drogowe w kolorze)

German Traffic Sign Recognition Benchmark (GTSRB) w wersji kolorowej to ten sam zestaw 43 klas z oficjalnym podziałem na trening i test [22], [42]. Na potrzeby eksperymentów obrazy są przeskalowane do 32×32 (ustawienia „cifarowe”) z zachowaniem trzech kanałów (RGB). Normalizacja wykonywana jest per kanał na zbiorze uczącym, a walidację wydzielono z części treningowej analogicznie jak dla wariantu Gray [46].

Wersja RGB została włączona, aby ocenić, w jakim stopniu informacja barwna może kompensować trudność związaną z rotacjami oraz na ile architektury rotacyjnie inwariantne (CyVGG/CyResNet) nadal poprawiają wyniki względem bazowych modeli (VGG/ResNet). Zastosowanie tych samych rozmiarów wejścia, tych samych podziałów oraz tego samego klasyfikatora pozwala na porównanie RGB i Gray w układzie 1:1.

W praktyce kolor stanowi silny sygnał (np. czerwone obramowania zakazów, żółte trójkąty ostrzegawcze, niebieskie nakazy), lecz nie eliminuje problemów wynikających z dużej zmienności punktu widzenia, skali, oświetlenia i rozmycia ruchu. Rotacje pozostają istotnym czynnikiem trudności, a informacja barwna pomaga głównie odróżniać klasy o podobnych kształtach.

W części eksperymentalnej stosowane są te same scenariusze kątowe co wcześniej: wariant bez rotacji jako punkt odniesienia, warianty z małymi i średnimi obrotami, połączenia różnych kombinacji kątów oraz pełny zakres $0-360^\circ$. Dzięki temu zachowana jest porównywalność między VGG/ResNet a CyVGG/CyResNet przy jednakowym budżecie obliczeń.

LEGO (obiekty 3D rzutowane na 2D)

Zbiór **Images of LEGO Bricks** (Kaggle) [43] obejmuje obrazy elementów LEGO renderowanych jako rzuty 2D. W tej pracy obrazy zostały skonwertowane do skali szarości i przeskalowane do 96×96 , aby zachować detale klocków. Ustalono **50 klas** (wejście 1-kanałowe), walidację wydzielono z części treningowej analogicznie jak w pozostałych zbiorach, a normalizacja jest liczona per kanał na zbiorze uczącym.

Wybór zbioru LEGO motywowany jest tym, że obiekty mają złożone kształty i drobne szczegóły, co stanowi naturalny test wrażliwości na orientację. W odróżnieniu od MNIST (proste cyfry) i GTSRB (silny sygnał koloru), LEGO lepiej izoluje **geometrię** obiektu, czyli układ wypustek i światłocien, dzięki czemu różnice między podejściem augmentacyjnym a architektonicznym (**CyVGG/CyResNet** vs **VGG/ResNet**) są czytelniejsze.

W eksperymentach zastosowano te same scenariusze kątowe co w innych zbiorach: wariant bez rotacji jako punkt odniesienia, warianty z małymi i średnimi obrotami, połączenia różnych kombinacji kątów oraz pełny zakres **$0-360^\circ$** . Porównania są prowadzone przy tej samej części klasyfikacyjnej i tym samym budżecie obliczeń, aby izolować wpływ komponentu rotacyjnego.

Przy przekształceniach log-polarnych i niewielkiej rozdzielczości rośnie gęstość próbkowania w pobliżu środka. W przetwarzaniu wstępnym stosowana jest interpolacja biliniarna i stały środek układu, co ogranicza artefakty i utrzymuje porównywalność między wariantami.

Sposób augmentacji danych: zakresy rotacji, łączenie zbiorów

Pipeline obsługuje dwa formaty wejścia. Pierwszy to klasyczny format IDX (ubyte), stosowany m.in. w zbiorze MNIST. Drugi to tryb NPY, w którym dane zapisywane są jako `train_images.npy` i `train_labels.npy`, zaś dla części testowej jako `test_images.npy` i `test_labels.npy`. Niezależnie od formatu zastosowana jest ta sama logika budowania zbiorów danych oraz ich podziału na `train` i `test`. Dodatkowo, w ścieżce IDX dla MNIST nazwa pliku `t10k` jest zamieniana na `test` przed uruchomieniem rotacji.

Sposób augmentacji danych: rotacje i łączenie zbiorów

Pipeline obsługuje dwa formaty wejścia. Pierwszy to format IDX (ubyte), stosowany m.in. w MNIST. Drugi to tryb NPY, w którym dane zapisywane są jako `train_images.npy` i `train_labels.npy`, a dla części testowej jako `test_images.npy` i `test_labels.npy`. Niezależnie od formatu stosowana jest ta sama logika budowania zbiorów oraz podziału na `train` i `test`.

Rotacje

Augmentacja rotacją występuje w dwóch wersjach. W pierwszej stosowane są kąty stałe: dla każdej z góry zadanej wartości tworzony jest osobny zestaw nazwany według szablonu `rotated-{theta}`. W praktyce wykorzystywane są dwie siatki kątów: co 30° (30, 60, ..., 330) oraz co 45° (45, 90, ..., 315). Dostępny jest również preset łączny `fixed_all`, który obejmuje te obie siatki. Dla każdej wartości kąta przygotowywane są oddzielne zbiory treningowe i testowe.

Druga wersja opiera się na przedziałach kątów. Tworzone są zbiory `rotated-a-b` dla dwunastu zakresów: [0,30), [30,60), ..., [330,360). Każdej próbce przypisywany jest losowy kąt z rozkładu jednostajnego w ramach danego przedziału. Losowanie odbywa się niezależnie dla każdej próbki i każdego przedziału, co zwiększa różnorodność danych.

Parametry przekształceń są stałe w obrębie formatu. W trybie NPY obrót wykonywany jest wokół środka kadru, z interpolacją liniową, bez rozszerzania płótna, a piksele wypadające poza obraz wypełniane są stałym kolorem tła. W trybie IDX używana jest funkcja `PIL.Image.rotate` w ustawieniach domyślnych, co utrzymuje stały rozmiar wyjściowy.

Łączenie zbiorów

Na podstawie zbiorów obrotowych tworzone są zbiory połączone. Zapisywane są one w folderze `merged_datasets/`, a ich nazwy zaczynają się od prefiksu `merged_`. Dla kątów stałych powstają zestawy `merged_fixed_30`, `merged_fixed_45` oraz `merged_fixed_all`. Dla wariantu przedziałowego dostępne są m.in. `merged_range_0_90`, `merged_range_90_180`, `merged_range_180_270`, `merged_range_270_360`, a także szersze `merged_range_0_180`, `merged_range_180_360` oraz pełny `merged_range_full_0_360`. Każdy z tych presetów może być rozszerzany o zbiór bez rotacji, co oznaczane jest dopiskiem `_plus_non_rotated`. Dla każdego presetu przygotowywane są osobno zbiory `train` i `test`.

Sposób łączenia zależy od formatu. W IDX pliki `*-images-idx3-ubyte` i `*-labels-idx1-ubyte`

są sklejane, a nagłówki aktualizowane są o nową liczbę próbek. W NPY wykonywana jest konkatenacja macierzy obrazów i wektorów etykiet wzdłuż osi próbek.

Organizacja katalogów

W katalogu bazowym znajduje się folder zbioru źródłowego, np. `dataset_X`, z plikami `train_images.npy`, `train_labels.npy`, `test_images.npy`, `test_labels.npy`. Obok tworzone są katalogi wariantów obrotowych, takie jak `rotated-30` czy `rotated-0-30`, z analogicznymi plikami dla podziałów `train` i `test`. Zbiory połączone zapisywane są w `merged_datasets/`, m.in. w `merged_fixed_30`, `merged_range_180_360_plus_non_rotated` oraz `merged_range_full_0_360_plus_non_rotated`, również z kompletami plików treningowych i testowych.

Scenariusze trenowanie - test

Do porównań wykorzystywany jest plik JSON z opisem scenariuszy ewaluacji. Nazwy w tym pliku odpowiadają ścieżkom na dysku. Przykładowe klucze (wartości mają tę samą postać) to: - `dataset_LEGO_non_rotated`,

- `merged_datasets/merged_fixed_30`,
- `merged_datasets/merged_fixed_30_plus_non_rotated`,
- `merged_datasets/merged_range_0_180`,
- `merged_datasets/merged_range_0_180_plus_non_rotated`,
- `merged_datasets/merged_range_180_360_plus_non_rotated`,
- `merged_datasets/merged_range_full_0_360_plus_non_rotated`,
- `rotated-30`,
- `rotated-45`,
- `rotated-0-30`,
- `rotated-90-120`.

Dla każdego zbioru treningowego przypisywana jest lista zbiorów testowych. Zawsze uwzględniany jest zbiór bazowy bez rotacji, sam zbiór treningowy oraz dodatkowe zbiory rotowane dobrane zgodnie z ustalonym limitem.

Architektury modeli

(VGG-E, ResNet-56, CyCNN)

Przegląd. W pracy wykorzystano bazowe architektury **VGG** (wariant **E**) i **ResNet** (wariant **56**) w ustawieniu dla **CIFAR** (obrazy 32×32). Warstwy splotowe to głównie 3×3 z padding=1. W VGG po każdym bloku stosowany jest `MaxPool2d(2)`, a w ResNecie rozdzielczość zmniejszana jest przez `stride=2`. Po części splotowej występuje **global average pooling (GAP)** oraz prosty **klasyfikator**. W VGG używana jest wersja z normalizacją (`VGG_bn`) oraz dwustopniowy klasyfikator $512 \rightarrow 512 \rightarrow C$ z dropoutem (w implementacji `AdaptiveAvgPool2d((1,1)) + nn.Sequential` z dwiema warstwami liniowymi i wyjściem C).

VGG - wariant E (VGG-19, „3×3 everywhere”)

Układ zgodny z [18], dostosowany do 32×32 (CIFAR). W plikach **VGG** i **CyVGG** konfiguracja **E** to: `[64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 512, 'M', 512, 512, 512, 512, 'M']`. Warstwy budowane są przez funkcję `make_layers(cfg['E'], ...)`, z opcją `BatchNorm(*_bn)` i z `MaxPool2d` wstawianym w miejscach oznaczonych symbolem 'M'. Za częścią splotową stosowane jest `AdaptiveAvgPool2d((1,1))`, następnie spłaszczenie (`flatten`) i dwie warstwy liniowe $512 \rightarrow 512 \rightarrow C$ z dropoutem.

- **Blok 1:** $64, 64 \rightarrow \text{MaxPool} (32 \rightarrow 16)$
- **Blok 2:** $128, 128 \rightarrow \text{MaxPool} (16 \rightarrow 8)$
- **Blok 3:** $256 \times 4 \rightarrow \text{MaxPool} (8 \rightarrow 4)$
- **Blok 4:** $512 \times 4 \rightarrow \text{MaxPool} (4 \rightarrow 2)$
- **Blok 5:** $512 \times 4 \rightarrow \text{MaxPool} (2 \rightarrow 1)$

W **CyVGG** każdą `Conv2d` zastąpiono **CyConv2d** (jest interfejs zgodny z `Conv2d`: 3×3 , padding=1, ze wsparciem dylacji ang. `stride`). Klasyfikator i układ bloków pozostają takie same jak w przypadku VGG.

ResNet-56 (CIFAR, $6n+2$ z $n=9$)

Schemat jak w [19] dla datasetu CIFAR. Na wejściu `Conv 3×3`, a dalej **3 grupy** po $n=9$ bloków `BasicBlock`. Grupa 1 (16 kanałów, `stride 1`), Grupa 2 (32 kanały, pierwszy blok `stride 2`), Grupa 3 (64 kanały, pierwszy blok `stride 2`). `BasicBlock` ma postać: `Conv3×3 → BN → ReLU → Conv3×3 → BN` + skrót.

Downsample - opcja A (CIFAR): podpróbkowanie przestrzenne `x[:, :, ::2, ::2]` i dopełnienie kanałów przez `F.pad(...)`. Zakończenie: **GAP** → warstwa liniowa $64 \rightarrow C$.

W **CyResNet** wszystkie `nn.Conv2d` zastąpiono **CyConv2d** (w tym `conv1` i konwolucje w `BasicBlock`). Pozostałe elementy (`BN`, `ReLU`, `shortcut`, `GAP`, `Linear`) pozostają bez zmian względem wersji bazowej.

Wersje cykliczne: CyVGG-E i CyResNet-56

Wersje cykliczne powstają przez zastąpienie każdej Conv2d warstwą CyConv2d. Interfejs (kernel size, stride, padding) jest zgodny z Conv2d, więc topologia sieci i część klasyfikacyjna pozostają bez zmian. CyConv2d opakowuje własną funkcję autograd (CyConv2dFunction) i wywołuje rozszerzenie CUDA CyConv2d_cuda.forward/backward(...). Wagi mają kształt [C_out, C_in, k, k] i są inicjalizowane przez xavier_uniform_. Moduł korzysta z dużego bufora roboczego na GPU (opisanego w kodzie jako „Workspace for Cy-Winograd algorithm”).

W definicjach modeli nie występuje jawna oś „orientacja” ani osobny pooling po orientacjach. Z punktu widzenia PyTorch parametry filtrów zachowują standardowy kształt [C_out, C_in, k, k]. Mechanizmy rotacyjne - o ile są użyte - realizowane są w jądrze CUDA CyConv2d_cuda, niewidocznym na poziomie kodu modeli. W praktyce inwariancja po stronie modeli nie jest wprowadzana osobno: GAP oraz ewentualne uśrednianie w klasyfikatorze działają tak samo jak w wersjach bazowych i nie ma dodatkowego „poolingu po orientacjach”.

Uzgodnienia I/O i selektor modeli

- **Kanały wejścia.** 1 kanał dla mnist, mnist-custom, GTSRB-custom, LEGO; 3 kanały w pozostałych przypadkach (np. CIFAR, GTSRB RGB).
- **Liczba klas.** MNIST/CIFAR-10: 10; GTSRB: 43; LEGO: 50; CIFAR-100: 100 - zgodnie z helperami get_num_classes.
- **Selektor modeli.** get_model(model, dataset, classify=True) zwraca jedną z wersji: vgg*, cyvgg*, resnet*, cyresnet*, zależnie od stringa modelu i nazwy zbioru danych.

Standardowe CNN

Jako bazy zastosowano **VGG** (wariant **E** / **VGG-19**) i **ResNet-56**. Konwolucje 3×3 z padding=1, w VGG z okresowym MaxPool2d(2), w ResNecie zaś jest zmniejszanie rozdzielczości przez stride=2. Po części spłotowej **GAP** i **klasyfikator** (w VGG są to dwie warstwy w pełni połączone 512→512→C z dropoutem, aż w ResNecie jest to Linear 64→C). Warianty VGG w kodzie występują także w wersjach *_bn (z BatchNorm). Modele te są z natury **ekwiwariantne translacyjnie** (dobrze znoszą przesunięcia), ale nie posiadają wbudowanego mechanizmu inwariancji względem rotacji, stąd stanowią więc punkt odniesienia dla wersji cyklicznych [18], [19].

Rotacyjnie inwariantne sieci (CyResNet, CyVGG)

Wersje cykliczne (**CyVGG-E**, **CyResNet-56**) powstały przez **podmianę każdej warstwy Conv2d na CyConv2d**. Architektura bloków, liczby kanałów, GAP i układ klasyfikatora pozostały bez zmian, co pozwala na porównanie wpływu samej warstwy spłotowej. W kodzie modeli **nie ma jawnego wymiaru orientacji** ani dedykowanego „poolingu po orientacjach”. Funkcjonalność związaną z rotacją zrealizowana została w jądrze CUDA z użyciem CyConv2d_cuda, wywoływanym z poziomu CyConv2d [4].

Transformacje polarne: linearpolar vs logpolar

Przekształcenie do układu biegunowego mapuje obraz z współrzędnych (x, y) na siatkę (ρ, ϕ) , gdzie ρ opisuje odległość od środka, a ϕ kąt. Dzięki temu obrót obrazu staje się przesunięciem wzdłuż osi ϕ , co upraszcza budowanie reprezentacji ekwiwariantnych względem rotacji. W praktyce stosowane są dwie siatki: linearpolar i logpolar.

W linearpolar przyrost po ρ i po ϕ jest równy w pikselach. Kąty są stabilne, implementacja prosta, a odwzorowanie dobrze nadaje się do budowania inwariancji rotacyjnej. Logpolar stosuje skalę logarytmiczną wzdłuż ρ , przez co zmiany skali w obrazie zamieniają się w przesunięcia po osi promienia. Takie odwzorowanie jest szczególnie użyteczne, gdy oprócz rotacji ważna jest również zmienność skali [20]. Wadą jest rosnąca gęstość próbkowania w pobliżu środka, co zwiększa wrażliwość na wybór punktu odniesienia i na szum.

Niezależnie od wariantu stosowana jest interpolacja biliniarna oraz cykliczne dopełnianie po ϕ , aby nie powstawały artefakty na brzegach kąta. Środek układu utrzymywany jest stały; w okolicy $\rho = 0$ warto wprowadzić wygładzenie lub pominąć kilka najbliższych próbek, by ograniczyć wpływ osobliwości i zniekształceń [4]. Wybór między linearpolar a logpolar zależy więc od celu: gdy kluczowa jest odporność na obrót, wystarcza siatka liniowa; gdy istotna jest także skala, korzystny bywa układ logarytmiczny kosztem większej troski o okolice środka.

Implementacja i środowisko eksperymentalne

Wszystkie eksperymenty realizowane zostały w środowisku **PyTorch** z rozszerzeniem CUDA dla warstwy cylindrycznej, dodatkowo zrobiona została automatyczna optymalizacja hiperparametrów w **Optunie** dla wybranych modeli (w tym referencyjnego) [7], [8]. Pipeline danych obejmuje warianty IDX i NPY oraz generator zbiorów rotowanych i zbiorów połączonych ze sobą(merged). Wyniki treningu oraz testów dla każdego modelu są zapisywane w formacie txt wraz z macierzami pomyłek w formatach png oraz npy. Następne dane są sprawdzane i automatycznie generowane są heatmaps train-test oraz ranking modeli. Zapisy metryk i konfiguracji z optuny trafiają do plików **CSV** i **JSON**, co ułatwia powtarzalność oraz porównywanie konfiguracji. Dodatkowo najlepsze checkpointy dla danego przypadku są zapisane jako modele już przetrenowane .pt. Wykorzystywany był optymalizator **SGD** wraz z *momentum* i *weight decay*. Zakresy i sposób doboru wartości hiperparametrów opisanostały w części poświęconej HPO.

Warstwa CyConv2d (CUDA) oraz jej implementacja

Warstwa CyConv2d korzysta z rozszerzenia C++/CUDA kompilowanego jako CyConv2d_cuda. Pliki źródłowe wykorzystywane do kompilacji to cycnn.cpp i cycnn_cuda.cu, ich budowanie przy użyciu setuptools z BuildExtension. Dzięki temu wywołania tej biblioteki z poziomu Pythona trafiają bezpośrednio do rdzeni CUDA. W pliku cycnn.cpp udostępnione są funkcje forward(...) i backward(...) z użyciem pybind11. Przyjmują one tensory input, weight oraz bufor workspace, a także parametry geometrii: stride, padding, dilation. Przed przekazaniem do implementacji CUDA sprawdzane jest, czy dane znajdują się na GPU i mają ciągły układ w pamięci. Następnie wywoływane są funkcje cyconv2d_cuda_forward i cyconv2d_cuda_backward.

Po stronie PyTorch warstwa jest opakowana w CyConv2dFunction z własnymi forward i backward. Moduł CyConv2d przechowuje wagi o kształcie [C_out, C_in, k, k] z inicjalizacją z użyciem xavier_uniform_. W metodzie forward wywoływana jest CyConv2dFunction.apply(...), do której przekazywane są parametry kroku, dopełnienia i dylacji oraz wskaźnik do bufora roboczego. Bufor workspace jest prealokowany na GPU jako tensor float32 o rozmiarze 1024*1024*1024 elementów, jest to w przybliżeniu około 4 GiB. W kodzie został opisany jako miejsce pracy wariantu algorytmu Winograda. Rozwiązanie to pozwala skrócić czas obliczeń, ale wymaga odpowiedniej ilości pamięci VRAM, przez co na kartach graficznych posiadających mniej VRAMU mogą pojawić się błędy OOM.

Integracja z modelami jest bezpośrednia. We wszystkich miejscach, gdzie w bazowych architekturach użyto nn.Conv2d, zarówno w conv1, jak i w konwolucjach wewnątrz bloków, w wersjach **CyVGG** i **CyResNet** wstawiono CyConv2d. Pozostałe elementy pozostają bez zmian: BatchNorm, ReLU, GAP i klasyfikator działają tak samo jak w wersjach referencyjnych. W samych definicjach modeli nie ma jawnej osi orientacji ani osobnego poolingu po orientacjach. Z punktu widzenia PyTorch wagi mają klasyczny kształt [C_out, C_in, k, k]. Jeśli pojawia się zachowanie związane z obrotami, to jest ono realizowane w kodzie CUDA (cycnn_cuda.cu), do którego odwołują się bindingi z cycnn.cpp.

Python, PyTorch

Środowisko jest oparte na **Pythonie** i **PyTorchu** z akceleracją CUDA. Modele definiowane są w stylu modułów `nn.Module`, zaś pętle uczące korzystają z klasycznego układu: forward, obliczenie straty, backward, aktualizacja optymalizatora. Zastosowane zostały usprawnienia backendowe (`torch.backends.cudnn.benchmark = True`, czyli ustanowienie F32 tam, gdzie jest to możliwe), co pozwala skrócić czas uczenia na nowszych GPU. W modelach jest używany optymalizator **SGD** z *momentum* i *weight decay*, a harmonogram uczenia opiera się na `ReduceLROnPlateau`. Kod modeli jest kompatybilny z ekosystemem i API PyTorch, więc warstwy bazowe można bez problemu wymieniać na odpowiedniki cylindryczne bez zmian w pozostałych fragmentach sieci neuronowej.

Struktura projektu

Projekt podzielony jest na dwie wzajemnie uzupełniające się części. Repozytorium treningowe zawiera modele (**VGG**, **ResNet** oraz **CyVGG**, **CyResNet**), warstwę `CyConv2d` z rozszerzeniem **C++/CUDA**, loader danych dla formatów **IDX** i **NPY**, transformacje i mapowania polarne oraz główny skrypt `main.py` oraz launchery dla poszczególnych datasetów. Znajdują się tam także pliki uruchamiające `Optunę` i konfiguracje eksperymentów.

Repozytorium zarządzające skupia narzędzia do przygotowania danych, rotacji i łączenia zbiorów, a także do analizy wyników. Dostępny jest interfejs CLI (`Typer`), który buduje zbiory, wczytuje logi i artefakty, zapisuje metryki do **SQLite** oraz generuje mapy ciepła train-test i zestawienia rankingowe. Artefakty są porządkowane w powtarzalnej strukturze katalogów: `logs/` dla przebiegów, `saves/` dla wag `.pt`, foldery z macierzami pomyłek w wariancie `.npz` i `.png`, a wyniki zbiorcze w `results/`.

Automatyzacja: skrypty trenowania, testowania, ewaluacji

Trenowanie modeli uruchamiane jest skryptem `launcher.py`, który wywołuje `main.py` z odpowiednimi parametrami dla danego modelu, zbioru, wariantu przekształceń i ustawień treningu. W trakcie ewaluacji zapisywana jest macierz pomyłek oraz podstawowe metryki dokładności. Repozytorium orkiestrujące dostarcza komendy CLI do pełnego przebiegu. Najpierw `preprocess` przygotowuje zbiory rotowane i zestawy połączone. Następnie uruchamiany jest trening i test, po czym `ingest` zbiera logi oraz wyniki i zapisuje je do bazy **SQLite**. Komenda `check-logs` weryfikuje kompletność przebiegów. Moduły `analyze` i `matrix-analyzer` tworzą mapy ciepła train-test, agregują macierze pomyłek, liczą statystyki i budują rankingi modeli. W wybranych konfiguracjach dołączona jest **Optuna**, która automatycznie stroi hiperparametry i zapisuje najlepsze konfiguracje wraz z wynikami do plików **CSV/JSON**.

Automatyczna optymalizacja hiperparametrów z wykorzystaniem Optuny

W celu poprawy jakości trenowanych modeli zastosowana została automatyczna optymalizacja hiperparametrów. Ręczne dobieranie wartości takich jak *learning rate*, *momentum* czy *weight decay* jest czasochłonne, podatne na błędy i bardzo często prowadzi do ustawień dalekich od optimum. Optymalne parametry zależą od architektury sieci, charakteru zbioru danych oraz użytych przekształceń polarnych. Z tego powodu wykorzystana została biblioteka **Optuna** [8], nowoczesne narzędzie do strojenia hiperparametrów (*Hyperparameter Optimization, HPO*).

W eksperymentach użyty został algorytm próbkowania **TPE (Tree structured Parzen Estimator)** oraz mechanizm **pruning Median**, który umożliwia wcześniejsze przerywanie prób o niskiej jakości. Dzięki temu czas potrzebnych obliczeń został skrócony nawet o trzydzieści procent. Wyniki każdej próby są zapisywane do plików **CSV** i **JSON**, co ułatwia późniejszą analizę oraz wierne odtworzenie najlepszych konfiguracji.

Proces optymalizacji uruchomiony został dla czterech modeli (**ResNet56**, **VGG19**, **CyResNet56**, **CyVGG19**) oraz dla dwóch wariantów transformacji (*linear polar*, *log polar*). Optymalizacja prowadzona była na zestawach oznaczonych jako *non_rotated*. W każdej konfiguracji wykonano 25 prób z limitem 10 epok na próbę, aby ograniczyć czas trwania eksperymentów. Przeszukiwane były następujące zakresy: *learning rate* od $1e-4$ do $5e-2$ w skali logarytmicznej, *weight decay* od $1e-7$ do $1e-3$ w skali logarytmicznej, *momentum* od 0.85 do 0.99 w skali liniowej.

Zastosowanie Optuny pozwoliło osiągnąć wyższą dokładność niż w ustawieniach dobieranych ręcznie. Najczęściej wybierane wartości *learning rate* mieściły się w przedziale od 0.001 do 0.01, co jest spójne z obserwacjami występującymi w literaturze. Korzyści były widoczne także w prostszych zbiorach, takich jak MNIST oraz GTSRB, a w bardziej złożonych scenariuszach optymalizacja ograniczyła liczbę arbitralnych decyzji i obniżyła koszt obliczeń.

Włączenie automatycznej optymalizacji do cyklu eksperymentalnego zwiększyło wiarygodność wyników. Otrzymane wyniki dla modeli oparte są na systematycznym dostrajaniu zgodnym z aktualnym stanem wiedzy, a nie ślepych na pojedynczych ręcznych próbach.

Obsługa GPU, Docker, WSL

Środowisko uruchomieniowe zostało zorganizowane tak, aby połączyć **wydajność GPU**, **powtarzalność kontenerów** oraz **spójność pracy w przypadku pracy na Windows przez WSL2**. Treningi i testy uruchamiane są w obrazie Dockera z przypiętymi wersjami **CUDA** i **cuDNN**, a ten sam obraz jest wykorzystywany w **WSL2**, co eliminuje różnice między stacjami roboczymi. Dane i artefakty (logi, checkpointy, macierze pomyłek) są montowane jako woluminy, więc katalogi z wynikami pozostają niezmiennie między sesjami i hostami. Dobór urządzeń kontrolowany jest przez **CUDA_VISIBLE_DEVICES**, a ścieżki wejścia i wyjścia mają jednolitą konwencję, co upraszcza późniejszą analizę i replikację. Monitorowanie obciążenia oraz VRAM odbywa się narzędziem **nvidia-smi**, natomiast szczegóły konfiguracji przepływu danych i precyzji obliczeń opisane zostały poniżej.

Obsługa GPU

Środowisko uruchomieniowe wykorzystuje akcelerację **CUDA** na kartach **RTX 3070 Ti 8 GB** oraz **RTX 3060 12 GB**. Włączone są optymalizacje backendowe: `torch.backends.cudnn.benchmark = True` oraz tryb **TF32** na architekturze Ampere. Transfery między CPU i GPU realizowane są z `pin_memory=True` w `DataLoader` oraz `non_blocking=True` przy kopiowaniu tensora na urządzenie, co zmniejsza narzut I/O. W miejscach, gdzie to bezpieczne do użycia, wykorzystywana jest **mieszana precyzja** z `torch.cuda.amp` i `GradScaler`. Monitorowanie obciążenia i pamięci odbywa się przez `nvidia-smi`. Należy uwzględnić wymóg pamięci VRAM dla warstwy `CyConv2d` (bufor roboczy ~4 GiB na GPU), co sprawia, że minimalna wymagana ilość VRAMu wynosi ~6GB.

Docker

Docker zapewnia powtarzalne środowisko z obsługą GPU. Obraz zawiera **PyTorch**, **CUDA**, **cuDNN** i zależności projektu. Uruchomienie odbywa się z **nvidia-container-toolkit**; woluminy mapują katalogi z danymi i wynikami:

```
docker run \
  --gpus all \
  -it --rm \
  -v /path/to/data:/workspace/data \
  -v /path/to/results:/workspace/results \
  cycnn:latest
```

WSL2

WSL2 służy do pracy w środowisku Windows z dostępem do tego samego obrazu Dockera i tej samej konfiguracji. Artefakty mogą być kopiowane po ścieżkach `\wsl$` do lokalnych katalogów celem analityki. Zachowana zostaje spójność nazw katalogów i plików, co ułatwia późniejszą ingestie do bazy **SQLite** i generowanie raportów. W przypadku różnic w separatorach wykorzystywanych do tworzenia ścieżek logika użytych narzędzi unika operacji zależnych od systemu, zaś ścieżki są przekazywane jawnie w CLI.

Organizacja logów, modeli, confusion matrixów

Artefakty przeprowadzonych eksperymentów są porządkowane w stałej strukturze katalogów. Dzięki temu skrypty analityczne mogą automatycznie odnajdywać logi, modele i macierze pomyłek, zestawiać wyniki i budować heatmapy train-test.

```
CyCNN-Enhanced-develop/
|-- ReadMe.md
|-- LICENSE
|-- .gitignore
|-- cycnn-extension/                # rozszerzenie C++/CUDA dla CyConv2d
|   |-- cycnn.cpp
|   |-- cycnn_cuda.cu
```

```

|   |-- setup.py
|-- cycnn/                                # trenowanie i testowanie modeli
|   |-- main.py
|   |-- utils.py
|   |-- data.py
|   |-- custom_loader.py
|   |-- image_transforms.py
|   |-- requirements.txt
|   |-- launcher_MNIST.py
|   |-- launcher_GTSRB.py
|   |-- launcher_GTSRB_RGB.py
|   |-- launcher_LEGO.py
|   |-- optuna_launcher.py
|   |-- optuna_driver_universal.py
|   |-- optuna_checker.py
|   |-- optuna_mnist.py
|   |-- train_test_scenarios_MNIST.json
|   |-- train_test_scenarios_GTSRB.json
|   |-- train_test_scenarios_GTSRB_RGB.json
|   |-- train_test_scenarios_LEGO.json
|   |-- models/
|       |-- getmodel.py
|       |-- cyconvlayer.py
|       |-- cyresnet.py
|       |-- cyvgg.py
|       |-- resnet.py
|       |-- vgg.py
|-- logs/                                # logi i (domyślnie) macierze pomyłek
|   |-- .gitkeep
|-- saves/                                # checkpointy (.pt)
|   |-- .gitkeep
|   |-- MNIST/                            # przykładowy podkatalog (opcjonalny)

```

Logi (logs/) zapisywane są podczas uruchomienia z flagą `--redirect`. Plik trafia na dysk pod wzorcem:

`cycnn/logs/<fname>.txt`

gdzie nazwa `<fname>` budowana jest z następujących składników:

`<dataset>-<model>[-<polar_transform>] [-<augmentation>] [-rotation_from_scenarios]`

Przykład używany w eksperymentach: `cycnn/logs/mnist-custom-cyresnet56-linearpolar_merged_datasets_merged_range_0_180_plus_non_rotated_train.txt`

Modele (saves/) zawiera on najlepsze checkpointy w formacie `.pt`. Zapis następuje przy poprawie wyniku lub zgodnie z polityką zapisu w skrypcie. Domyślna ścieżka ma postać:

```
cycnn/saves/<fname>.pt
```

Jeśli podano `--model-save-path`, checkpoint zostaje zapisany dokładnie w tej ścieżce, z pominięciem wzorca domyślnego.

Macierze pomyłek (PNG i NPY) zapisywane są razem z wynikiem testu. Gdy nie podano `--output-dir`, używany jest katalog zależny od pary train–test:

```
cycnn/logs/<train_set>_test_on_<test_set>/  
    confusion_matrix.npy  
    confusion_matrix.png
```

W przypadku, gdy jest podany parametr `--output-dir <DIR>`, pliki trafiają do wskazanej lokalizacji:

```
<DIR>/confusion_matrix.npy  
<DIR>/confusion_matrix.png
```

Scenariusze train–test (JSON) zawierają gotowe listy par zbiorów, których nazwy odpowiadają rzeczywistym ścieżkom na dysku. W repozytorium znajdują się następujące scenariusze:

```
cycnn/train_test_scenarios_MNIST.json  
cycnn/train_test_scenarios_GTSRB.json  
cycnn/train_test_scenarios_GTSRB_RGB.json  
cycnn/train_test_scenarios_LEGO.json
```

Eksperymenty

Scenariusze trenowania/testowania (opis JSON)

Scenariusze zdefiniowane są przez pliki JSON, który mapują **zestawy treningowy** na listę **zestawów testowych**. Klucze i wartości są po prostu ścieżkami katalogów w strukturze danych. Dzięki temu w łatwy sposób powiązane zostały nazwy w pliku JSON z realnymi ścieżkami na dysku i można było przeprowadzić serię eksperymentów bez ręcznej konfiguracji czy uruchamiania.

Przykładowy fragment JSON:

```
{
  "dataset_LEGO_non_rotated": [
    "dataset_LEGO_non_rotated",
    "merged_datasets/merged_fixed_30",
    "merged_datasets/merged_fixed_45_plus_non_rotated",
    "rotated-90",
    "rotated-90-120"
  ],
  "merged_datasets/merged_range_full_0_360_plus_non_rotated": [
    "dataset_LEGO_non_rotated",
    "merged_datasets/merged_range_0_180",
    "merged_datasets/merged_range_180_360",
    "rotated-120",
    "rotated-210-240"
  ]
}
```

Listy testowe budowane zostały według określonych reguł ich generatorowania. Zawsze zawierają one co najmniej zestaw bazowy bez rotacji oraz sam zestaw treningowy. Pozostałe pozycje dobierane są z puli wariantów obrotowych i presetów łączonych do ustalonego limitu. Wygenerowane nazwy są zgodne z konwencjami katalogów, które powstają podczas preprocessingu.

Podczas procedury treningu-testowania pętla uruchomieniowa odczytuje scenariusz, a następnie wykonuje następującą sekwencję: trening na danym **train_set**, a następnie testy na wszystkich **test_set** z listy. Poniżej uproszczony szkic logiki, którą realizują skrypty:

```
for train_set in scenarios:
    model = build_model(...)
    fit(model, train_set, ...)
    for test_set in scenarios[train_set]:
        acc, cm = evaluate(model, test_set)
        save_metrics_and_artifacts(acc, cm, train_set, test_set, model)
```

Takie podejście porządkuje eksperymenty. Pozwala też tworzyć mapy „trenuj na X, testuj na Y” oraz automatycznie budować rankingi modeli wspólne dla całego zbioru scenariuszy.

Pomiar skuteczności (accuracy, macierze pomyłek)

Śledzenie metryk: średnia, mediana, odchylenie standardowe

Analiza skuteczności względem rotacji

Ranking modeli

Porównanie wyników

CyCNN vs klasyczne CNN

VGG vs CyVGG

Resnet vs CyResNet

CyVGG vs CyResNet

[...] cyresnet56 uczył się dłużej niż cyvgg19, ale dawał bardziej stabilne wyniki, w szczególności w przypadku funkcji aktywacji typu logpolar. I jak to się mówi - nie można zjeść ciastka i mieć go też (ang. „You can’t eat your cake and have it too” [47]).

Wpływ transformacji (linearpolar vs logpolar)

Wydajność na różnych zbiorach

Wnioski

Skuteczność rotacyjnych architektur

Wnioski z automatyzacji i systematyzacji ewaluacji

Propozycje dalszych badań

Aneks

Listingi kodów

Dodatkowe wykresy, tablice wyników

Bibliografia

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: <https://www.deeplearningbook.org/>
- [2] V. Dumoulin and F. Visin, “A guide to convolution arithmetic for deep learning.” 2016. Available: <https://arxiv.org/abs/1603.07285>
- [3] T. Cohen and M. Welling, “Group equivariant convolutional networks,” in *Proceedings of the 33rd international conference on machine learning (ICML)*, 2016. Available: <https://proceedings.mlr.press/v48/cohen16.html>
- [4] J. Kim, W. Jung, H. Kim, and J. Lee, “CyCNN: A rotation invariant CNN using polar mapping and cylindrical convolution layers,” *arXiv preprint arXiv:2007.10588*, 2020, Available: <https://arxiv.org/abs/2007.10588>
- [5] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in neural information processing systems 32 (NeurIPS)*, 2019. Available: <https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>
- [6] Python Software Foundation, “Python 3 documentation.” 2025. Available: <https://docs.python.org/3/>
- [7] PyTorch Core Team, “PyTorch documentation (stable).” 2025. Available: <https://pytorch.org/docs/stable/>
- [8] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, ACM, 2019, pp. 2623–2631. doi: 10.1145/3292500.3330701.
- [9] NVIDIA Corporation, “CUDA toolkit documentation.” 2025. Available: <https://docs.nvidia.com/cuda/>
- [10] NVIDIA Corporation, “NVIDIA cuDNN developer guide.” 2025. Available: <https://docs.nvidia.com/deeplearning/cudnn/developer-guide/index.html>
- [11] “NVIDIA tensor cores.” 2024. Available: <https://developer.nvidia.com/tensor-cores>
- [12] “NVIDIA ampere architecture: TF32 tensor cores.” 2024. Available: <https://developer.nvidia.com/blog/tf32-matmul>
- [13] P. Micikevicius *et al.*, “Mixed precision training,” in *International conference on learning representations (ICLR)*, 2018.
- [14] “Automatic mixed precision (AMP) - PyTorch docs.” 2024. Available: <https://pytorch.org/docs/stable/amp.html>
- [15] “Ubuntu documentation.” 2024. Available: <https://help.ubuntu.com/>
- [16] “Windows subsystem for linux (WSL) documentation.” 2024. Available: <https://learn.microsoft.com/windows/wsl/>
- [17] Docker, Inc., “Docker documentation.” 2025. Available: <https://docs.docker.com/>
- [18] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014, Available: <https://arxiv.org/abs/1409.1556>

- [19] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2016, pp. 770–778. Available: https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html
- [20] B. S. Reddy and B. N. Chatterji, “An FFT-based technique for translation, rotation and scale-invariant image registration,” *IEEE Transactions on Image Processing*, vol. 5, no. 8, pp. 1266–1271, 1996, doi: 10.1109/83.506761.
- [21] Y. LeCun, C. Cortes, and C. J. C. Burges, “The MNIST database of handwritten digits.” 1998. Available: <http://yann.lecun.com/exdb/mnist/>
- [22] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, “The german traffic sign recognition benchmark: A multi-class classification competition,” in *Proceedings of the international joint conference on neural networks (IJCNN)*, 2011, pp. 1453–1460. doi: 10.1109/IJCNN.2011.6033395.
- [23] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask r-CNN,” in *Proceedings of the IEEE international conference on computer vision (ICCV)*, 2017. Available: https://openaccess.thecvf.com/content_ICCV_2017/papers/He_Mask_R-CNN_ICCV_2017_paper.pdf
- [24] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *MICCAI*, in LNCS, vol. 9351. 2015, pp. 234–241. Available: https://link.springer.com/chapter/10.1007/978-3-319-24574-4_28
- [25] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004, Available: <https://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf>
- [26] M. Jaderberg, K. Simonyan, A. Zisserman, and K. Kavukcuoglu, “Spatial transformer networks,” in *Advances in neural information processing systems (NIPS)*, 2015. Available: <https://proceedings.neurips.cc/paper/2015/file/33ceb07bf4eeb3da587e268d663aba1a-Paper.pdf>
- [27] C. Esteves, C. Allen-Blanchette, A. Makadia, and K. Daniilidis, “Learning SO(3)-equivariant representations with spherical CNNs,” in *Proceedings of the european conference on computer vision (ECCV)*, 2018, pp. 52–68. Available: https://openaccess.thecvf.com/content_ECCV_2018/html/Carlos_Esteves_Learning_SO3_Equivariant_ECCV_2018_paper.html
- [28] R. Hartley and A. Zisserman, *Multiple view geometry in computer vision*, 2nd ed. Cambridge University Press, 2004.
- [29] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *Proceedings of the 37th international conference on machine learning (ICML)*, PMLR, 2020. Available: <https://proceedings.mlr.press/v119/chen20j/chen20j.pdf>
- [30] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum contrast for unsupervised visual representation learning,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2020. Available: https://openaccess.thecvf.com/content_CVPR_2020/papers/He_Momentum_Contrast_for_Unsupervised_Visual_Representation_Learning_CVPR_2020_paper.pdf

- [31] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: Bandit-based configuration evaluation for hyperparameter optimization,” *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018, Available: <https://jmlr.org/papers/volume18/16-558/16-558.pdf>
- [32] D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” in *International conference on learning representations (ICLR)*, 2019. Available: <https://web.engr.oregonstate.edu/~tgd/publications/hendrycks-dietterich-benchmarking-neural-network-robustness-to-common-corruptions-and-perturbations-iclr2019.pdf>
- [33] T. DeVries and G. W. Taylor, “Improved regularization of convolutional neural networks with cutout.” 2017. Available: <https://arxiv.org/abs/1708.04552>
- [34] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [35] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković, “Geometric deep learning: Grids, groups, graphs, geodesics, and gauges,” *arXiv preprint arXiv:2104.13478*, 2021, Available: <https://arxiv.org/abs/2104.13478>
- [36] A. Azulay and Y. Weiss, “Why do deep convolutional networks generalize so poorly to small image transformations?” in *Journal of vision*, 2019, pp. 1–14.
- [37] W. Luo, Y. Li, R. Urtasun, and R. Zemel, “Understanding the effective receptive field in deep convolutional neural networks,” in *Advances in neural information processing systems (NeurIPS)*, 2016.
- [38] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *International conference on machine learning (ICML)*, 2015.
- [39] M. Lin, Q. Chen, and S. Yan, “Network in network,” in *International conference on learning representations (ICLR)*, 2014.
- [40] M. Weiler and G. Cesa, “General $e(2)$ -equivariant steerable CNNs,” *arXiv preprint arXiv:1911.08251*, 2019, Available: <https://arxiv.org/abs/1911.08251>
- [41] T. S. Cohen, M. Geiger, and M. Weiler, “A general theory of equivariant CNNs on homogeneous spaces,” in *Advances in neural information processing systems (NeurIPS)*, 2019. Available: <https://papers.neurips.cc/paper/9114-a-general-theory-of-equivariant-cnns-on-homogeneous-spaces.pdf>
- [42] “GTSRB dataset — official page and citation.” Accessed: Aug. 26, 2025. [Online]. Available: https://benchmark.ini.rub.de/gtsrb_dataset.html
- [43] J. Hazelzet, “Images of LEGO bricks.” Accessed: Aug. 26, 2025. [Online]. Available: <https://www.kaggle.com/datasets/joosthazelzet/lego-brick-images>
- [44] Y. LeCun and C. Cortes, “MNIST handwritten digit database.” Accessed: Aug. 26, 2025. [Online]. Available: <https://yann.lecun.org/exdb/mnist/>
- [45] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in neural information processing systems (NeurIPS)*, 2019. Available: <https://pytorch.org/>
- [46] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, “Man vs. Computer: Benchmarking machine learning algorithms for traffic sign recognition,” *Neural Networks*, vol. 32, pp. 323–332, 2012, doi: 10.1016/j.neunet.2012.02.016.

- [47] T. J. Kaczynski, “Industrial society and its future,” *The Washington Post*, Sep. 1995, Available: <https://www.washingtonpost.com/wp-srv/national/longterm/unabomber/manifesto.text.htm>